

Assessment - Customer Complaint & Escalation Management System

Duration	Total Marks	Assessment Type	Primary Stack
1 Day (8 hours)	100	Individual POC	Python + FastAPI + SQL + LLM + Streamlit

1. Overview

Build a customer complaint and escalation management system. Customers should submit complaints and track their own cases; support agents should work assigned complaints, add notes and resolve issues; managers should monitor queues, SLA breaches, escalations and team performance; administrators should configure complaint categories, channels, SLA policies and support teams. Complaint priority, SLA deadlines, escalation rules, duplicate detection, ownership and analytics must be implemented in Python/SQL. The LLM is used only for a complaint-trend/escalation narrative generated from already-computed metrics, with an optional individual complaint summary if time permits.

GenAI note: Use Azure OpenAI/OpenAI/Microsoft Foundry if available. If closed-source/cloud access is not available, use an appropriate open-source/local model. Do not make access to a paid provider a dependency for completing the assessment.

Each complaint should include at minimum: id, customer_id, product_id, category, channel, subject, description, priority, status, assigned_team_id, assigned_agent_id, created_at, sla_due_at, resolved_at and closed_at. Each escalation should include at minimum: id, complaint_id, level, reason, escalated_at, escalated_by, assigned_to and resolved_at.

2. Minimum Technical Architecture

Layer	Minimum Requirement	Accepted Options
Application Language	Python application with domain/service/repository layers	Python 3.10+
Backend API	FastAPI REST API with async endpoints/dependencies where I/O is involved	FastAPI + Uvicorn
Frontend	Streamlit application consuming FastAPI over HTTP	Streamlit + httpx/requests
Authentication	Secure password hashing + JWT bearer authentication	pwdlib/passlib/bcrypt + PyJWT/python-jose
Authorization	Role-based and ownership/resource-level checks	FastAPI dependencies + service/repository policy checks
Relational Data Store	SQL database with ORM, relationships, transactions, constraints and indexes	SQLite + SQLAlchemy/SQLModel; PostgreSQL accepted
Async / Parallel / Jobs	Async DB/model I/O, asyncio.gather() for independent operations, and tracked background jobs	AsyncSession, async HTTP client, asyncio.gather, Semaphore, BackgroundTasks/task runner
In-Memory Cache	Process-local TTL cache for frequently requested analytics/dashboard data with explicit invalidation and HIT/MISS visibility	Custom cache service or cachetools.TTLCache; Redis optional
Data Exchange	At least one validated import/export workflow	CSV and/or JSON
LLM	One text-generation model called directly from Python for the required AI feature	Azure OpenAI/OpenAI/Foundry; Ollama/Hugging Face/local model accepted if cloud access is unavailable
Configuration	Secrets, JWT settings and model/database settings outside source code	.env + .env.example / environment variables

3. Functional Requirements

A. Complaint, Category & SLA CRUD

- Create, list, retrieve and update complaints; customers can cancel/close only when allowed by workflow.
- Create/manage complaint categories, channels, support teams and SLA policies.
- Assign complaints to agents/teams and record internal notes, customer-visible updates and escalation history.
- Validate category, priority, complaint status and escalation-level enums.
- Support filtering by customer, product, category, channel, status, priority, assigned team/agent, SLA state and date range.
- Support sorting and simple pagination.

B. Python Business Rules

Rules must be implemented in normal Python services/classes.

- Implement a complaint workflow such as NEW -> ASSIGNED -> IN_PROGRESS -> RESOLVED -> CLOSED, with REOPENED allowed only under defined rules.
- Calculate SLA due time from complaint category/priority/channel and determine due-soon/breached status.
- Calculate deterministic priority/severity from configurable business rules; the LLM must not be the sole source of priority.
- Automatically determine escalation level when SLA breach thresholds are crossed; managers can also perform permitted manual escalation.
- Detect potential duplicate complaints from the same customer/product/category within a configurable period and prevent/flag duplicates according to policy.
- Enforce complaint ownership and team scope so agents cannot modify complaints outside their assignment unless permitted.
- Prevent invalid transitions such as CLOSED -> IN_PROGRESS without reopening and RESOLVED -> ASSIGNED without a defined reopen path.
- Invalidate affected analytics caches when complaint status, priority, assignment or escalation changes.

C. Python Analytics

- Complaint volume by category, product, channel and priority for a selected period.
- SLA compliance percentage and count of due-soon/breached complaints by team.
- Average first-response and resolution turnaround time.
- Escalation rate and reopen rate by category/team.
- Agent workload: assigned open complaints, resolved count and average resolution time.
- Top recurring complaint categories/products and aging of unresolved complaints.
- Do not ask the LLM to calculate deterministic metrics.

D. Import / Export

- Import historical complaints or customer/product master data from CSV/JSON with row-level validation.
- Report accepted/rejected records with reasons and deterministic duplicate handling.
- Export filtered complaint register, SLA breach report or escalation report to CSV/JSON.

E. Direct LLM Feature

- Implement POST /ai/complaint-trend-summary for a selected team/product/category/date range.
- First compute complaint volume, top categories, SLA compliance, escalation rate, reopen rate and turnaround metrics in Python/SQL.
- Pass only those computed values plus optional manager instructions to the LLM.
- Ask the model to generate a concise customer-support trend narrative and 2-3 operational observations.
- Optional: POST /ai/complaint-summary can summarize one complaint thread, but it must not replace priority/SLA/escalation rules.
- Return controlled errors/fallback behavior when the model provider is unavailable.

F. Authentication & Authorization

- Implement registration/login with securely hashed passwords and JWT bearer tokens.

- Support at least the following roles: ADMIN, SUPPORT_MANAGER, SUPPORT_AGENT, CUSTOMER.
- CUSTOMER can create/view only their own complaints and permitted customer-visible updates.
- SUPPORT_AGENT can work complaints assigned to them or their permitted queue and add internal notes/status updates.
- SUPPORT_MANAGER can assign/escalate complaints, view team complaints and access team analytics.
- ADMIN can configure categories, SLA policies, support teams and organization-wide access.
- Enforce role, ownership and resource-scope checks in API/service/repository logic rather than relying only on hidden Streamlit controls.
- Return 401 for missing/invalid tokens and 403 for authenticated users who lack permission.

G. Async, Parallel Processing, Caching & Background Jobs

- Use async database sessions/repositories for API I/O and an async model/provider client where supported.
- Move bulk historical complaint import and complaint/SLA report generation to tracked background jobs that return job_id immediately.
- Expose GET /jobs/{job_id} with PENDING, RUNNING, COMPLETED and FAILED states plus progress, timestamps, result summary and error details.
- Use asyncio.to_thread or equivalent when synchronous file/CPU work would otherwise block an async route.
- Cache GET /analytics/support-dashboard and GET /analytics/sla-performance for a short TTL such as 30-120 seconds. Cache keys must include user/role and normalized filters where relevant.
- Invalidate affected cache entries after writes or successful background imports that change the underlying analytics.
- Log or expose cache HIT/MISS so the evaluator can verify behavior.
- Use asyncio.gather() to build a composite dashboard response from at least three independent operations, such as queue counts, SLA breach metrics, agent workload, turnaround statistics. Each gathered database coroutine must use its own AsyncSession/unit of work.
- Background import/export/report jobs must return HTTP 202 + job_id and track progress (0-100), started_at, finished_at, accepted/rejected counts or result location, and error details.

H. Advanced Python & SQL

- Implement a reusable SLA calculator and complaint-aging utility in Python.
- Implement deterministic duplicate detection using customer_id + product/category + normalized subject/date window or external_id.
- Model customers, products, complaint_categories, complaints, complaint_notes/messages, escalations, support_teams/users and jobs relationally with foreign keys, constraints and indexes.
- Provide a JOIN + GROUP BY query showing complaint volume, SLA compliance and escalation count per category/team for a selected period.
- Provide at least one CTE/window-function report, for example RANK() OVER (PARTITION BY team ORDER BY resolved_count DESC) for agent performance or aging rank of unresolved complaints.
- Use transactions for multi-row imports and workflow operations that update complaint state plus escalation/audit records.

I. Streamlit Frontend

- Provide login/logout using FastAPI authentication and store the JWT token in st.session_state.
- Provide a customer complaint submission/history view.
- Provide support-agent work queue with status/update/resolution actions.
- Provide manager assignment/escalation views and support analytics dashboard.
- Allow background import/export job submission and polling plus the AI complaint-trend-summary action.
- All business data must be accessed through FastAPI. Streamlit must not read or write the database directly.

4. API Contract

Endpoint	Input	Output	Core Concept
POST /auth/register	email/password/role	user	authentication
POST /auth/login	email/password	JWT token	authentication
GET /auth/me	bearer token	current user	authorization
POST /complaint-categories	category/SLA rules	created category	CRUD + SQL constraints
POST /complaints	complaint fields	created complaint	CRUD + business rules
GET /complaints	filters/sort/page	authorized complaint list	Python/SQL filtering
GET /complaints/{id}	complaint id	complaint + timeline	ownership/authorization
PATCH /complaints/{id}/assign	team/agent	updated complaint	assignment policy
PATCH /complaints/{id}/status	status/notes	updated complaint	workflow + cache invalidation
POST /complaints/{id}/escalate	level/reason	escalation record	SLA/business rules
POST /complaints/{id}/notes	visibility/text	note	audit + ownership
GET /analytics/support-dashboard	filters	cached composite metrics	Python + SQL + TTL + gather
GET /analytics/sla-performance	team/category/date	SLA rows	JOIN + GROUP BY
GET /analytics/agent-ranking	team/date	ranked agents	CTE/window SQL
POST /complaints/import-job	CSV/JSON	job_id	background processing
POST /complaints/export-job	filters/format	job_id	background processing
GET /jobs/{job_id}	job id	status/progress/result/error	async/background jobs
POST /ai/complaint-trend-summary	team/product/category/date + instruction	LLM summary	direct async LLM generation
GET /health	none	service/model readiness	API quality

5. Implementation Constraints & Deliverables

Implementation Constraints

- Priority, SLA deadlines/breaches, escalation rules, duplicate detection, workflow transitions and support analytics must be computed in Python/SQL, not by the LLM.
- No hard-coded API keys or secrets; include .env.example and configuration validation.
- Supply at least 12 sample customers, 3 products, 4 complaint categories and 35 complaints across multiple statuses/priorities with escalation examples.
- Use relational SQL through SQLAlchemy/SQLModel; JSON-only persistence is not sufficient.
- LLM output must be generated from live computed support analytics, not hard-coded text.
- Passwords must be hashed; JWT/model/database settings remain outside source code.
- Database constraints/indexes should support customer/product/category/status/priority/SLA/assignment filters used by common queries.
- At least one advanced SQL JOIN/aggregate and one CTE/window-function report are mandatory.
- Streamlit must call FastAPI rather than directly accessing the database.
- TTL cache is mandatory for support analytics; SQL remains the source of truth.
- At least one endpoint must use asyncio.gather() for three or more independent I/O operations with separate DB sessions/unit-of-work objects.
- Background import/export/report jobs must expose 202 + job_id, progress, timestamps, result/error and failure behavior.
- Use proper Python logging with meaningful context; print-only logging is not accepted.
- Important modules/classes/functions must use type hints and meaningful docstrings.

Submission Deliverables

- Structured source code with route/service/repository/configuration separation.
- README.md with setup, architecture, run commands, model/provider choice, assumptions, error handling and API examples.
- requirements.txt or pyproject.toml and .env.example.
- Seed/sample customer, product, category, complaint, escalation and user data.
- CSV/JSON import sample file.
- pytest/API/Postman/curl/Swagger evidence.
- At least two support-dashboard results and two AI complaint-trend summaries.
- Streamlit application with authenticated customer/agent/manager/admin workflows.
- Seed/demo users for all roles.
- SQL/ORM schema with relationships, constraints/indexes and evidence of advanced queries.
- Background job demo with success and simulated failure.
- Cache MISS/HIT/invalidation evidence.
- Concurrency demo showing `asyncio.gather()` rather than `serial awaits`.

6. Marking Scheme - 100 Marks

Section	Assessment Criteria	Marks	Evidence Expected
1. Advanced Python & Code Quality	Typed modular Python, reusable utilities, standard-library features where appropriate, custom exceptions, logging, docstrings and clean configuration.	20	Substantial Python logic is visible outside route handlers; code is typed, modular, testable and maintainable.
2. OOP, Business Logic, Async, Caching & Background Processing	Service/repository design, non-trivial domain rules, <code>asyncio.gather</code> parallel I/O, TTL caching, tracked background jobs, job-state handling and defensive coding.	15	Business rules are correct, cache behavior is measurable, parallel async execution is safe, and background workflows are observable.
3. SQL, Data Handling & Persistence	Normalized relational schema, ORM relationships, transactions, constraints/indexes, advanced SQL, import/export and analytics.	20	Relational data is correct and at least one JOIN/aggregation plus one CTE/window-function query is demonstrated.
4. FastAPI, Authentication & Authorization	API design, JWT login, secure password hashing, bearer flow, RBAC/ownership checks, validation and meaningful error handling.	20	Protected endpoints correctly distinguish unauthenticated, forbidden and permitted requests; validation/errors are consistent.
5. Streamlit Frontend	Login/session handling, role-aware UI, operational forms, dashboards and background-job/LLM interaction through FastAPI.	10	A usable Streamlit client consumes the API instead of bypassing the backend.
6. LLM Model Integration	Direct model call, prompt quality, dynamic computed input, structured parsing where applicable and graceful provider failures/fallback.	10	The LLM feature works from live Python/SQL-computed complaint/SLA data, provides a useful support narrative, and does not replace deterministic priority, workflow or escalation rules.
7. Testing & Documentation	Reproducible setup, pytest/API tests, auth/SQL/background-job coverage and clear README/API examples.	5	Evaluator can run, verify and understand the complete POC quickly.

7. Minimum Pass / Acceptance Checklist

- [] FastAPI starts successfully and GET /health reports readiness.
- [] JWT auth and CUSTOMER/SUPPORT_AGENT/SUPPORT_MANAGER/ADMIN authorization work with correct 401/403 behavior.
- [] Complaint/category/note/escalation workflows work against persistent SQL data.
- [] Customers cannot access another customer's complaints; agents are correctly scoped to permitted work.
- [] SLA due/breach calculations and complaint aging are correct.
- [] Invalid status transitions and unauthorized assignment/escalation attempts are rejected.
- [] Duplicate complaint handling works deterministically.
- [] CSV/JSON import/export is functional with accepted/rejected details.
- [] Required JOIN/aggregate and CTE/window SQL reports are demonstrated and correct.

- [] Background jobs expose 202 + job_id, progress, timestamps, result/error and FAILED behavior.
- [] Support analytics show cache MISS/HIT and invalidation after writes.
- [] Composite support dashboard uses asyncio.gather() for at least three independent operations.
- [] A real LLM/local model is called for complaint-trend summary and fails gracefully when unavailable.
- [] Streamlit provides role-aware customer, agent and manager workflows.
- [] Logging, meaningful exceptions, type hints, docstrings, tests and README are present.

8. Suggested Evaluator Test Cases

1. Create complaints for multiple customers/products/categories and verify filtering/pagination.
2. Attempt to access another customer's complaint and verify 403/not-found according to policy.
3. Verify valid NEW -> ASSIGNED -> IN_PROGRESS -> RESOLVED -> CLOSED workflow.
4. Attempt invalid CLOSED -> IN_PROGRESS transition without reopen and verify rejection.
5. Create complaints with different priorities/categories and verify SLA due calculations.
6. Advance/mock time or use test data to verify due-soon/breached state and escalation rules.
7. Create potential duplicate complaints and verify deterministic duplicate handling.
8. Import a mixed valid/invalid complaint file and inspect accepted/rejected details.
9. Import the same data twice and verify idempotent/duplicate behavior.
10. Validate SLA-performance JOIN/GROUP BY report against manually calculated data.
11. Validate the window/CTE agent-ranking or aging report.
12. Request the same support dashboard twice and verify MISS then HIT; change status/escalation and verify invalidation.
13. Mock three independent dashboard operations with delays and verify asyncio.gather() parallel execution.
14. Submit import/export background job and verify 202, progress, timestamps and completion.
15. Simulate background-job failure and verify FAILED state and continued API responsiveness.
16. Verify role behavior for CUSTOMER, SUPPORT_AGENT, SUPPORT_MANAGER and ADMIN.
17. Request AI complaint-trend summary and confirm all reported figures match Python/SQL analytics.
18. Use invalid model configuration and verify controlled fallback/error plus internal ERROR logging.